

App DD 2025 V3



À implémenter

-  Envoi mail automatique après cocher présent/absent (à traiter)



CAHIER DES CHARGES COMPLET

Derviche Diffusion - Plateforme de Réservation Professionnelle

Version : 3.0

Date : 27 novembre 2025

Statut : En cours



SOMMAIRE

1. Présentation du Projet
2. Architecture Technique
3. Modèle de Données
4. Parcours Utilisateurs
5. Fonctionnalités par Rôle
6. Règles Métier
7. Interfaces & Wireframes
8. Sécurité & RGPD
9. Roadmap & Priorisation
10. Annexes Techniques

1. PRÉSENTATION DU PROJET

1.1 Contexte

Derviche Diffusion est une société de diffusion de spectacles vivants qui représente des compagnies artistiques (théâtre, danse, marionnettes, cirque, etc.) auprès des programmeurs de salles de théâtre.

Tout au long de l'année, Derviche Diffusion organise des représentations professionnelles pour permettre aux programmeurs de découvrir les spectacles et potentiellement les programmer dans leurs salles.

1.2 Objectif de la Plateforme

Créer un outil de **gestion de réservations professionnelles** permettant aux programmeurs de :

- Découvrir les spectacles programmés
- Réserver gratuitement des places (ou payantes sur place)
- Gérer leurs réservations en autonomie

Et permettant à **Derviche Diffusion** de :

- Gérer l'ensemble de la programmation
- Suivre les réservations en temps réel
- Accueillir et qualifier les professionnels sur place
- Analyser la fréquentation et les profils

1.3 Utilisateurs Cibles

Rôle	Nombre estimé	Objectif
Derviche Diffusion (Super-Admin)	2-3 personnes	Gérer la plateforme, les utilisateurs et la configuration globale
Derviche Diffusion (Admin)	3-7 personnes	Gérer les spectacles et accueillir les professionnels
Externes Derviche (Externe-DD)	10-20 personnes	Accueillir sur place et gérer les réservations de spectacles assignés
Programmateurs (Professionnels)	500-1000	Découvrir et réserver des spectacles
Compagnies Artistiques	15-20	Consulter les réservations de leurs spectacles

1.4 URLs

- **Production** : derviche-pro.vercel.app
 - **Staging** : derviche-pro-staging.vercel.app
-

2. ARCHITECTURE TECHNIQUE

2.1 Stack Technologique

Frontend

Framework: Next.js 14+ (App Router)
Language: TypeScript
Styling:

- Tailwind CSS
- shadcn/ui (composants)

State Management:

- React Query (cache & sync serveur)
- Context API (state local)

Forms: React Hook Form + Zod

Backend

BaaS: Supabase

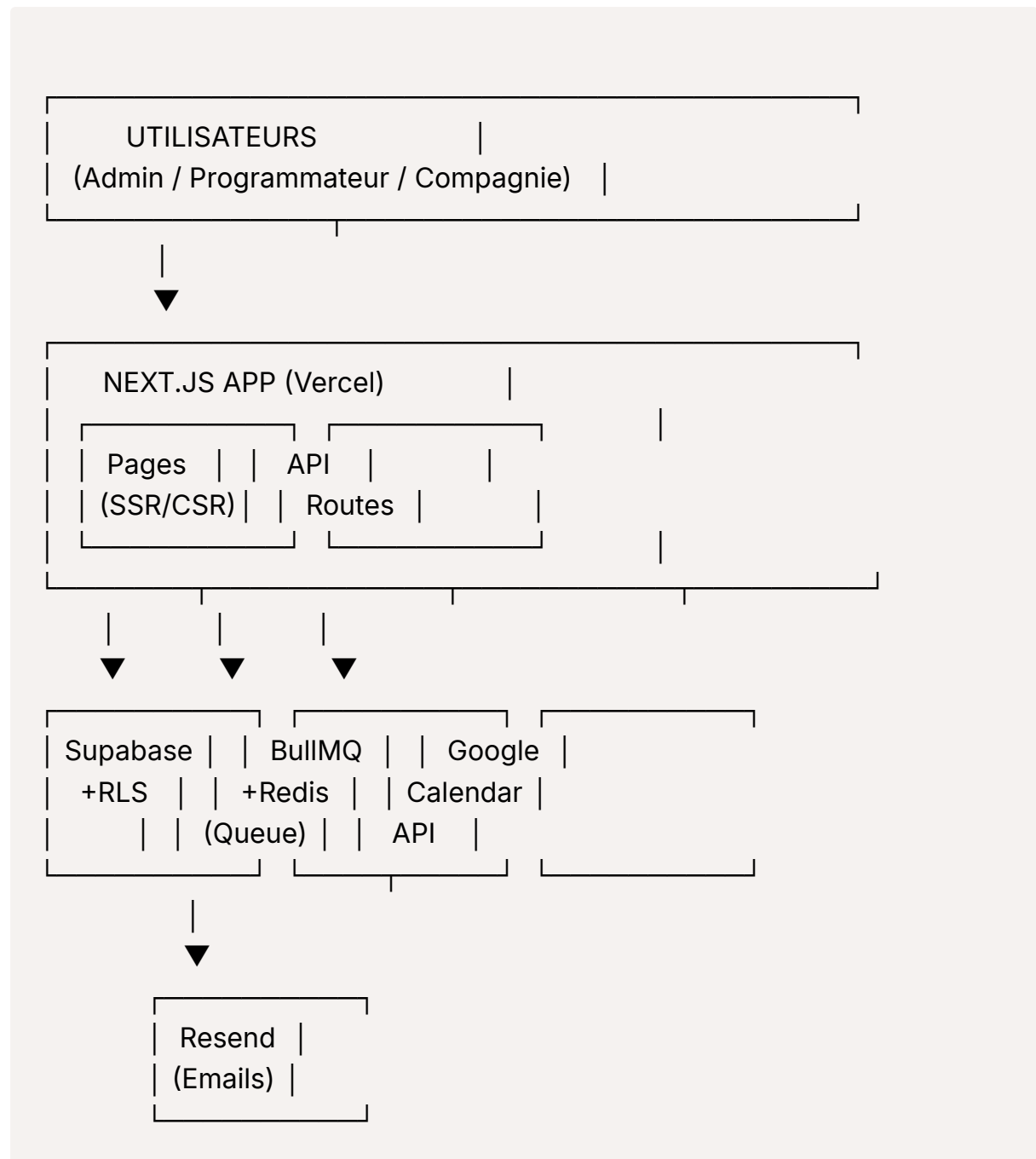
- PostgreSQL (base de données)
- Auth (authentification)
- Storage (fichiers)
- Row Level Security (RLS)

API: Next.js API Routes (serverless)
Queue: BullMQ + Redis (Upstash)

Services Externe

Email: Resend
Calendar: Google Calendar API
Deployment: Vercel
CDN: Vercel Edge Network

2.2 Schéma d'Architecture



2.3 Flux de Données Critiques

Création d'une Réservation

1. User soumet formulaire
2. Validation Zod côté client
3. POST /api/reservations

4. Transaction Supabase :
 - INSERT reservation (status: confirmed)
 - UPDATE slot (remaining_capacity - num_places)
5. Ajout de 3 jobs dans la queue BullMQ :
 - createGoogleCalendarEvent
 - sendConfirmationEmail (programmeur)
 - sendNotificationEmail (compagnie/derviche)
6. Réponse immédiate : { success: true, reservationId }
7. Workers traitent les jobs en arrière-plan (2-5 sec)
8. Programmeur reçoit email + événement Google Calendar

Modification d'une Réservation

1. User modifie (créneau, places, ou infos)
2. PUT /api/reservations/[id]
3. Transaction Supabase :
 - UPDATE reservation
 - Ajuster remaining_capacity (ancien + nouveau créneau)
4. Jobs queue :
 - updateGoogleCalendarEvent
 - sendModificationEmail (si demandé)
5. Réponse + notification

2.4 Décisions Techniques Validées

Question	Décision	Justification
Gestion quotas Google Calendar	Queue BullMQ + Redis	Robustesse, gestion des pics, retry automatique
Plusieurs réservations	Événements Calendar séparés	Simplicité, modification indépendante
Check-in modif profil	Mise à jour directe du profil	Simplicité, données à jour pour prochaines fois
Recherche catalogue	Front-end (JavaScript)	40-50 spectacles, pas besoin de back-end complexe
Interface mobile check-in	PWA (Progressive Web App)	Installable, expérience native, même codebase

Question	Décision	Justification
Mode offline check-in	✗ Reporté à Phase 2	Complexité supplémentaire, non critique pour MVP

2.5 Progressive Web App (PWA)

Objectif : Fournir une expérience mobile native pour le check-in sans passer par les stores.

Caractéristiques

- **Installable** sur l'écran d'accueil (iOS + Android)
- **Icône de l'app** avec le logo de l'organisation
- **Expérience fullscreen** (pas de barre d'adresse)
- **Même codebase** que la version desktop
- **Mises à jour automatiques** (pas de soumission aux stores)
- **Offline** : reporté à Phase 2 (synchronisation différée)

Configuration technique

```
// next.config.js avec next-pwa
{
  "name": "Derviche Pro Check-in",
  "short_name": "Check-in",
  "icons": [
    {
      "src": "/icon-192×192.png",
      "sizes": "192×192",
      "type": "image/png"
    },
    {
      "src": "/icon-512×512.png",
      "sizes": "512×512",
      "type": "image/png"
    }
  ],
  "start_url": "/",
  "display": "standalone",
}
```

```
"background_color": "#ffffff",  
"theme_color": "#000000"  
}
```

2.6 Architecture des Routes par Rôle

Routes Admin & Externe-DD

- Connexion : `/admin/login`
- Dashboard : `/admin`
- **Check-in** : `/admin/checkin`
- Gestion spectacles : `/admin/shows`
- Réservations : `/admin/reservations`
- Paramètres : `/admin/settings`

Routes Compagnies

- Connexion : `/company/login`
- Dashboard : `/company`
- **Check-in** : `/company/checkin`
- Réservations : `/company/reservations`
- Statistiques : `/company/stats`

Routes Programmeurs

- Connexion : `/login`
- Dashboard : `/professional`
- Catalogue : `/shows`
- Mes réservations : `/professional/reservations`

Logique de Redirection Automatique

Détection du rôle + device au login :

```
// Pseudo-code de redirection post-login  
const role = user.role;
```

```

const isMobile = window.innerWidth < 768;

if (isMobile) {
  // Mobile → Redirection vers check-in
  if (role === 'company') {
    router.push('/company/checkin');
  } else if (['admin', 'super-admin', 'externe-dd'].includes(role)) {
    router.push('/admin/checkin');
  } else if (role === 'professional') {
    router.push('/professional');
  }
} else {
  // Desktop → Dashboard normal
  if (role === 'company') {
    router.push('/company');
  } else if (['admin', 'super-admin', 'externe-dd'].includes(role)) {
    router.push('/admin');
  } else if (role === 'professional') {
    router.push('/professional');
  }
}

```

Filtrage des Créneaux par Rôle

Admin & Externe-DD (`/admin/checkin`) :

- **Admin** : Voit tous les créneaux
- **Externe-DD** : Voit uniquement les créneaux de ses spectacles assignés (via `user_show_assignments`)
- Filtrage automatique côté RLS

Compagnies (`/company/checkin`) :

- Voit uniquement les créneaux de **leurs spectacles**
- Filtre supplémentaire : uniquement les créneaux où `hosted_by='company'`
- Filtrage automatique côté RLS

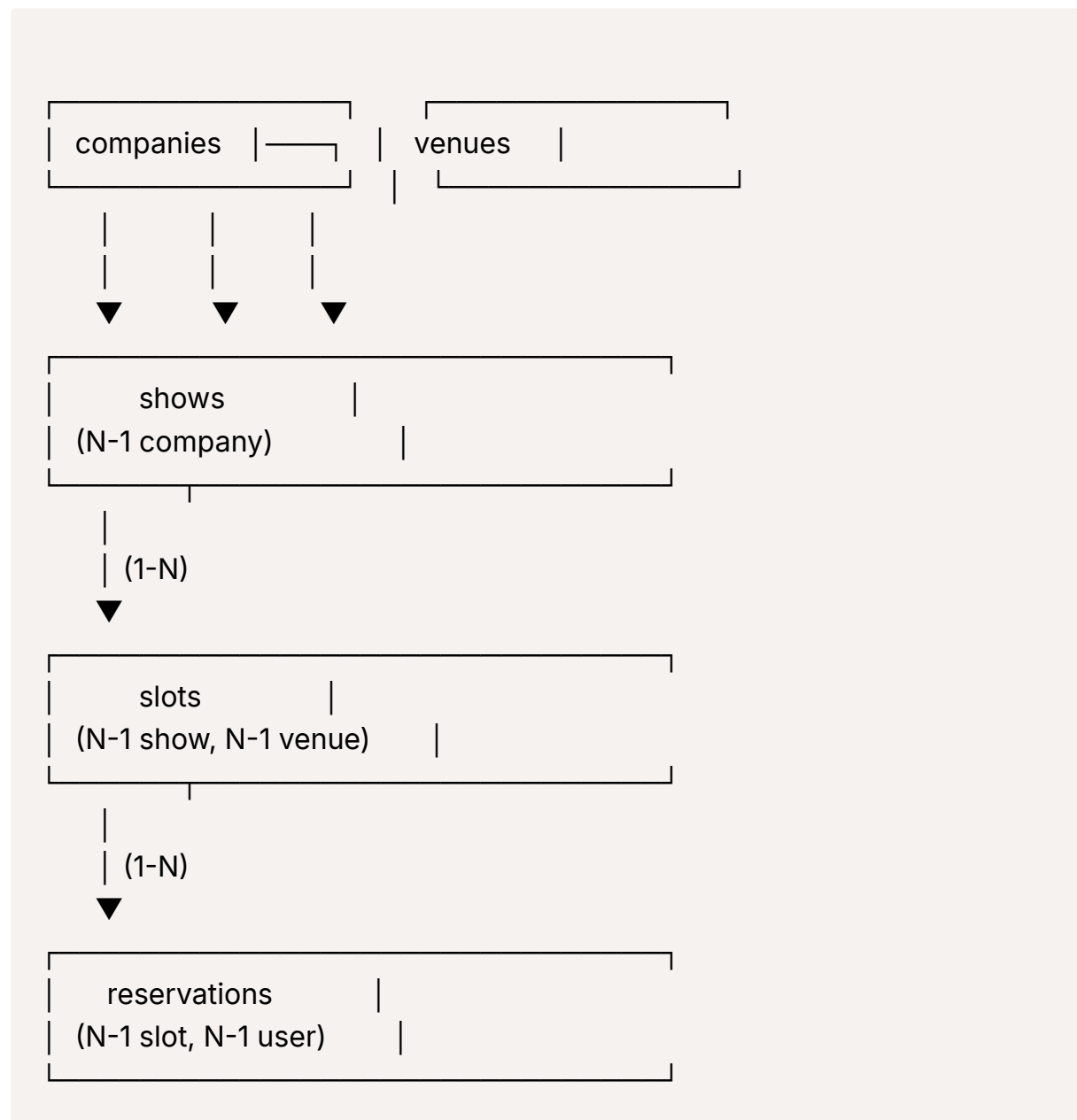
Gestion des Sessions

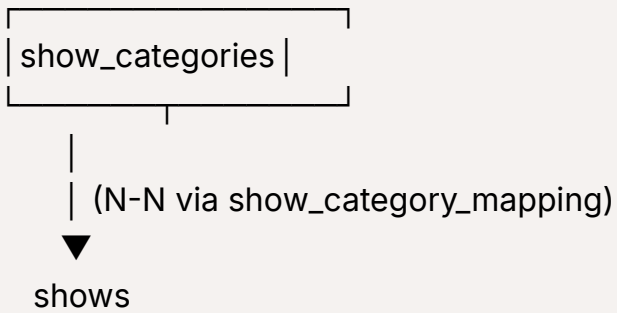
Supabase Auth avec refresh automatique :

- **Durée token JWT** : ~7 jours
- **Refresh automatique** : Géré par le client Supabase
- **Stockage** : localStorage + cookies sécurisés
- **Persistence** : L'utilisateur reste connecté entre les sessions
- **Déconnexion** : Manuelle via bouton ou expiration après inactivité prolongée

3. MODÈLE DE DONNÉES

3.1 Diagramme Entité-Relations





3.2 Tables Principales

profiles

```
CREATE TABLE profiles (  
  id UUID PRIMARY KEY REFERENCES auth.users(id),  
  email TEXT UNIQUE NOT NULL,  
  first_name TEXT,  
  last_name TEXT,  
  phone TEXT,  
  email2 TEXT,  
  phone2 TEXT,  
  function TEXT,  
  structure TEXT,  
  afc_number TEXT,  
  address TEXT,  
  comments TEXT,  
  company_id UUID REFERENCES companies(id) ON DELETE SET NULL,  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  updated_at TIMESTAMPTZ DEFAULT NOW()  
);  
  
-- Index pour lien company  
CREATE INDEX idx_profiles_company ON profiles(company_id);
```

user_roles

```
CREATE TABLE user_roles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
  role TEXT NOT NULL CHECK (role IN ('super-admin', 'admin', 'professiona
l', 'company', 'externe-dd')),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(user_id, role)
);
```

user_show_assignments

```
CREATE TABLE user_show_assignments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  show_id UUID NOT NULL REFERENCES shows(id) ON DELETE CASCADE,
  assigned_by UUID REFERENCES profiles(id) ON DELETE SET NULL,
  assigned_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(user_id, show_id)
);
```

-- Index pour performances

```
CREATE INDEX idx_user_show_assignments_user ON user_show_assignme
nts(user_id);
CREATE INDEX idx_user_show_assignments_show ON user_show_assignm
ents(show_id);
```

companies

```
CREATE TABLE companies (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  contact_email TEXT NOT NULL,
  contact_phone TEXT,
  website TEXT,
  description TEXT,
  logo_url TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
```

```
updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

venues

```
CREATE TABLE venues (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  address TEXT NOT NULL,
  city TEXT NOT NULL,
  postal_code TEXT NOT NULL,
  country TEXT DEFAULT 'France',
  latitude DECIMAL(10, 8),
  longitude DECIMAL(11, 8),
  description TEXT,
  contact_email TEXT,
  contact_phone TEXT,
  photo_url TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

show_categories

```
CREATE TABLE show_categories (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL UNIQUE,
  slug TEXT NOT NULL UNIQUE,
  icon TEXT,
  display_order INTEGER DEFAULT 0,
  description TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

shows

```

CREATE TABLE shows (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  slug TEXT NOT NULL UNIQUE,
  title TEXT NOT NULL,
  company_id UUID NOT NULL REFERENCES companies(id),
  short_description TEXT,
  long_description TEXT,
  duration_minutes INTEGER,
  image_url TEXT,
  gallery_urls TEXT[],
  status TEXT NOT NULL DEFAULT 'draft' CHECK (status IN ('draft', 'published', 'archived')),
  price_type TEXT NOT NULL DEFAULT 'free' CHECK (price_type IN ('free', 'paid_on_site')),
  price_amount DECIMAL(10, 2),
  max_reservations_per_booking INTEGER DEFAULT 4 CHECK (max_reservations_per_booking BETWEEN 1 AND 10),
  practical_info TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Index pour performances
CREATE INDEX idx_shows_company ON shows(company_id);
CREATE INDEX idx_shows_status ON shows(status);
CREATE INDEX idx_shows_slug ON shows(slug);

```

show_category_mapping

```

CREATE TABLE show_category_mapping (
  show_id UUID REFERENCES shows(id) ON DELETE CASCADE,
  category_id UUID REFERENCES show_categories(id) ON DELETE CASCADE,
  PRIMARY KEY (show_id, category_id)
);

```

slots

```

CREATE TABLE slots (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  show_id UUID NOT NULL REFERENCES shows(id) ON DELETE CASCADE,
  venue_id UUID NOT NULL REFERENCES venues(id),
  date DATE NOT NULL,
  time TIME NOT NULL,
  capacity INTEGER NOT NULL CHECK (capacity > 0),
  remaining_capacity INTEGER NOT NULL CHECK (remaining_capacity >=
0),
  hosted_by TEXT NOT NULL CHECK (hosted_by IN ('derviche', 'compan
y')),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),

  -- Contrainte d'unicité
  UNIQUE(show_id, venue_id, date, time)
);

-- Index pour performances
CREATE INDEX idx_slots_show ON slots(show_id);
CREATE INDEX idx_slots_venue ON slots(venue_id);
CREATE INDEX idx_slots_date ON slots(date);
CREATE INDEX idx_slots_hosted_by ON slots(hosted_by);

```

reservations

```

CREATE TABLE reservations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  slot_id UUID NOT NULL REFERENCES slots(id) ON DELETE CASCADE,
  user_id UUID REFERENCES profiles(id) ON DELETE SET NULL,

  -- Données invité (si réservation sans compte)
  guest_first_name TEXT,
  guest_last_name TEXT,
  guest_email TEXT,
  guest_phone TEXT,
  guest_function TEXT,

```

```

guest_structure TEXT,
guest_afc_number TEXT,

-- Données réservation
num_places INTEGER NOT NULL CHECK (num_places > 0),
status TEXT NOT NULL DEFAULT 'confirmed' CHECK (status IN ('confirmed', 'cancelled', 'no_show')),
special_requests TEXT,

-- Check-in
checkin_status TEXT CHECK (checkin_status IN ('present_loved', 'present_press', 'present_neutral', 'absent')),
checkin_comment TEXT,
checkin_venue_notes TEXT,
checkin_internal_notes TEXT,
checkin_at TIMESTAMPTZ,
checkin_by UUID REFERENCES profiles(id) ON DELETE SET NULL,

-- Google Calendar
google_calendar_event_id TEXT,

-- Timestamps
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW(),
cancelled_at TIMESTAMPTZ,
cancellation_reason TEXT,

-- Contraintes
CHECK (user_id IS NOT NULL OR (guest_email IS NOT NULL AND guest_firstname IS NOT NULL AND guest_lastname IS NOT NULL))
);

-- Index pour performances
CREATE INDEX idx_reservations_slot ON reservations(slot_id);
CREATE INDEX idx_reservations_user ON reservations(user_id);
CREATE INDEX idx_reservations_guest_email ON reservations(guest_email);
CREATE INDEX idx_reservations_status ON reservations(status);

```

```
CREATE INDEX idx_reservations_checkin_status ON reservations(checkin_status);
CREATE INDEX idx_reservations_created_at ON reservations(created_at);
```

app_settings

```
CREATE TABLE app_settings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  key TEXT NOT NULL UNIQUE,
  value JSONB NOT NULL,
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

-- Valeurs par défaut

```
INSERT INTO app_settings (key, value) VALUES
('organization_name', '"Derviche Pro"'),
('logo_url', 'null'),
('contact_email', 'null'),
('contact_phone', 'null'),
('website', 'null'),
('rate_limit_auth_login', '10'),
('rate_limit_auth_magic_link', '6'),
('rate_limit_auth_password_reset', '6'),
('rate_limit_reservations_create', '20'),
('rate_limit_reservations_update', '40'),
('rate_limit_reservations_delete', '40'),
('rate_limit_emails_manual', '100'),
('rate_limit_catalog_api', '200'),
('rate_limit_catalog_pages', '400'),
('rate_limit_admin_operations', '100'),
('rate_limit_admin_exports', '20');
```

sent_notifications

```
CREATE TABLE sent_notifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  reservation_id UUID REFERENCES reservations(id) ON DELETE CASCADE,
```



```

type TEXT NOT NULL CHECK (type IN ('confirmation', 'modification', 'cancellation', 'reminder_7d', 'reminder_2d', 'reminder_12h', 'checkin_update')),
recipient_email TEXT NOT NULL,
sent_at TIMESTAMPTZ DEFAULT NOW(),
email_provider_id TEXT,
error TEXT
);

-- Index pour performances
CREATE INDEX idx_sent_notifications_reservation ON sent_notifications(reservation_id);
CREATE INDEX idx_sent_notifications_type ON sent_notifications(type);
CREATE INDEX idx_sent_notifications_sent_at ON sent_notifications(sent_at);

```

audit_logs

```

CREATE TABLE audit_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES profiles(id) ON DELETE SET NULL,
  action TEXT NOT NULL CHECK (action IN ('create', 'update', 'delete', 'login', 'logout', 'checkin')),
  entity_type TEXT NOT NULL CHECK (entity_type IN ('show', 'slot', 'reservation', 'user', 'company', 'venue', 'category')),
  entity_id UUID,
  details JSONB,
  ip_address TEXT,
  user_agent TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Index pour performances
CREATE INDEX idx_audit_logs_user ON audit_logs(user_id);
CREATE INDEX idx_audit_logs_entity ON audit_logs(entity_type, entity_id);
CREATE INDEX idx_audit_logs_action ON audit_logs(action);
CREATE INDEX idx_audit_logs_created_at ON audit_logs(created_at);

```

3.3 Fonctions Base de Données

```
-- Vérifier si un utilisateur a un rôle spécifique
CREATE OR REPLACE FUNCTION has_role(role_name TEXT)
RETURNS BOOLEAN AS $$
BEGIN
    RETURN EXISTS (
        SELECT 1 FROM user_roles
        WHERE user_id = auth.uid()
        AND role = role_name
    );
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- Vérifier si un utilisateur est super-admin
CREATE OR REPLACE FUNCTION is_super_admin()
RETURNS BOOLEAN AS $$
BEGIN
    RETURN EXISTS (
        SELECT 1 FROM user_roles
        WHERE user_id = auth.uid()
        AND role = 'super-admin'
    );
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- Vérifier si un utilisateur est admin ou super-admin
CREATE OR REPLACE FUNCTION is_admin_or_super()
RETURNS BOOLEAN AS $$
BEGIN
    RETURN EXISTS (
        SELECT 1 FROM user_roles
        WHERE user_id = auth.uid()
        AND role IN ('admin', 'super-admin')
    );
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

```

-- Vérifier si un externe-DD a accès à un spectacle
CREATE OR REPLACE FUNCTION externe_has_access_to_show(show_id_param
UUID)
RETURNS BOOLEAN AS $$
BEGIN
    RETURN EXISTS (
        SELECT 1 FROM user_show_assignments
        WHERE user_id = auth.uid()
        AND show_id = show_id_param
    );
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- Fonction pour mettre à jour automatiquement remaining_capacity
CREATE OR REPLACE FUNCTION update_slot_capacity()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        UPDATE slots
        SET remaining_capacity = remaining_capacity - NEW.num_places
        WHERE id = NEW.slot_id;
    ELSIF TG_OP = 'UPDATE' THEN
        -- Si on change de créneau
        IF OLD.slot_id != NEW.slot_id THEN
            -- Libérer l'ancien créneau
            UPDATE slots
            SET remaining_capacity = remaining_capacity + OLD.num_places
            WHERE id = OLD.slot_id;
            -- Réserver le nouveau
            UPDATE slots
            SET remaining_capacity = remaining_capacity - NEW.num_places
            WHERE id = NEW.slot_id;
        -- Si on change le nombre de places
        ELSIF OLD.num_places != NEW.num_places THEN
            UPDATE slots
            SET remaining_capacity = remaining_capacity + OLD.num_places - NEW.num_places
            WHERE id = NEW.slot_id;
        
```

```

    END IF;
ELSIF TG_OP = 'DELETE' THEN
    -- Libérer la capacité si annulation
    IF OLD.status != 'cancelled' THEN
        UPDATE slots
        SET remaining_capacity = remaining_capacity + OLD.num_places
        WHERE id = OLD.slot_id;
    END IF;
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger pour la gestion automatique des capacités
CREATE TRIGGER manage_slot_capacity
AFTER INSERT OR UPDATE OR DELETE ON reservations
FOR EACH ROW EXECUTE FUNCTION update_slot_capacity();

```

3.4 Row Level Security (RLS)

```

-- =====
-- PROFILES
-- =====
ALTER TABLE profiles ENABLE ROW LEVEL SECURITY;

-- Tout le monde peut voir son propre profil
CREATE POLICY "users_own_profile"
ON profiles FOR SELECT
TO authenticated
USING (id = auth.uid());

-- Tout le monde peut mettre à jour son propre profil
CREATE POLICY "users_update_own_profile"
ON profiles FOR UPDATE
TO authenticated
USING (id = auth.uid());

-- Super-admin et admin peuvent voir tous les profils

```

```

CREATE POLICY "admin_all_profiles"
  ON profiles FOR SELECT
  TO authenticated
  USING (is_admin_or_super());

-- Super-admin et admin peuvent modifier les profils (pour check-in)
CREATE POLICY "admin_update_profiles"
  ON profiles FOR UPDATE
  TO authenticated
  USING (is_admin_or_super());

-- Externe-DD peut voir et modifier les profils (pour check-in de leurs spect
acles)
CREATE POLICY "externe_dd_profiles"
  ON profiles FOR ALL
  TO authenticated
  USING (has_role('externe-dd'));

-- =====
-- USER_ROLES
-- =====
ALTER TABLE user_roles ENABLE ROW LEVEL SECURITY;

-- Les utilisateurs peuvent voir leur propre rôle
CREATE POLICY "users_own_role"
  ON user_roles FOR SELECT
  TO authenticated
  USING (user_id = auth.uid());

-- Super-admin peut tout gérer
CREATE POLICY "super_admin_all_roles"
  ON user_roles FOR ALL
  TO authenticated
  USING (is_super_admin())
  WITH CHECK (is_super_admin());

-- =====
-- USER_SHOW_ASSIGNMENTS

```

```

-- =====
ALTER TABLE user_show_assignments ENABLE ROW LEVEL SECURITY;

-- Super-admin : tout voir et tout gérer
CREATE POLICY "super_admin_all_assignments"
  ON user_show_assignments FOR ALL
  TO authenticated
  USING (is_super_admin())
  WITH CHECK (is_super_admin());

-- Externe-DD : voir uniquement ses propres assignments
CREATE POLICY "externe_dd_own_assignments"
  ON user_show_assignments FOR SELECT
  TO authenticated
  USING (has_role('externe-dd') AND user_id = auth.uid());

-- =====
-- SHOWS
-- =====
ALTER TABLE shows ENABLE ROW LEVEL SECURITY;

-- Public : voir les spectacles publiés
CREATE POLICY "public_published_shows"
  ON shows FOR SELECT
  TO authenticated, anon
  USING (status = 'published');

-- Admin : tout voir et tout gérer
CREATE POLICY "admin_all_shows"
  ON shows FOR ALL
  TO authenticated
  USING (is_admin_or_super())
  WITH CHECK (is_admin_or_super());

-- Externe-DD : voir uniquement les spectacles assignés
CREATE POLICY "externe_dd_assigned_shows"
  ON shows FOR SELECT
  TO authenticated

```

```

    USING (has_role('externe-dd') AND externe_has_access_to_show(id));

-- Compagnies : voir leurs propres spectacles
CREATE POLICY "company_own_shows"
  ON shows FOR SELECT
  TO authenticated
  USING (
    has_role('company')
    AND company_id IN (
      SELECT id FROM companies WHERE id = company_id
    )
  );

-- =====
-- SLOTS
-- =====
ALTER TABLE slots ENABLE ROW LEVEL SECURITY;

-- Public : voir les créneaux des spectacles publiés
CREATE POLICY "public_slots"
  ON slots FOR SELECT
  TO authenticated, anon
  USING (
    EXISTS (
      SELECT 1 FROM shows
      WHERE shows.id = slots.show_id AND shows.status = 'published'
    )
  );

-- Admin : tout voir et tout gérer
CREATE POLICY "admin_all_slots"
  ON slots FOR ALL
  TO authenticated
  USING (is_admin_or_super())
  WITH CHECK (is_admin_or_super());

-- Externe-DD : voir uniquement les créneaux de leurs spectacles assignés
CREATE POLICY "externe_dd_assigned_slots"

```

```

ON slots FOR SELECT
TO authenticated
USING (
    has_role('externe-dd')
    AND externe_has_access_to_show(show_id)
);

-- Compagnies : voir les créneaux de leurs spectacles
CREATE POLICY "company_own_slots"
ON slots FOR SELECT
TO authenticated
USING (
    has_role('company')
    AND EXISTS (
        SELECT 1 FROM shows
        WHERE shows.id = slots.show_id
        AND shows.company_id = (SELECT company_id FROM profiles WHERE
id = auth.uid())
    )
);

-- =====
-- RESERVATIONS
-- =====
ALTER TABLE reservations ENABLE ROW LEVEL SECURITY;

-- Public : créer une réservation
CREATE POLICY "public_create_reservations"
ON reservations FOR INSERT
TO authenticated, anon
WITH CHECK (true);

-- Utilisateurs : voir leurs propres réservations
CREATE POLICY "users_own_reservations"
ON reservations FOR SELECT
TO authenticated
USING (user_id = auth.uid());

```



```

-- Utilisateurs : modifier leurs propres réservations
CREATE POLICY "users_update_own_reservations"
  ON reservations FOR UPDATE
  TO authenticated
  USING (user_id = auth.uid());

-- Admin : tout voir et tout gérer
CREATE POLICY "admin_all_reservations"
  ON reservations FOR ALL
  TO authenticated
  USING (is_admin_or_super())
  WITH CHECK (is_admin_or_super());

-- Externe-DD : voir et gérer les réservations de leurs spectacles assignés
CREATE POLICY "externe_dd_assigned_reservations"
  ON reservations FOR ALL
  TO authenticated
  USING (
    has_role('externe-dd')
    AND EXISTS (
      SELECT 1 FROM slots
      JOIN user_show_assignments ON slots.show_id = user_show_assignme
nts.show_id
      WHERE slots.id = reservations.slot_id
      AND user_show_assignments.user_id = auth.uid()
    )
  );

-- Compagnies : voir les réservations de leurs spectacles
CREATE POLICY "company_own_reservations"
  ON reservations FOR SELECT
  TO authenticated
  USING (
    has_role('company')
    AND EXISTS (
      SELECT 1 FROM slots
      JOIN shows ON slots.show_id = shows.id
      WHERE slots.id = reservations.slot_id

```

```

        AND shows.company_id = (SELECT company_id FROM profiles WHERE
id = auth.uid())
    )
);

-- Compagnies : modifier check-in de leurs spectacles
CREATE POLICY "company_checkin_reservations"
ON reservations FOR UPDATE
TO authenticated
USING (
    has_role('company')
    AND EXISTS (
        SELECT 1 FROM slots
        JOIN shows ON slots.show_id = shows.id
        WHERE slots.id = reservations.slot_id
        AND shows.company_id = (SELECT company_id FROM profiles WHERE
id = auth.uid())
        AND slots.hosted_by = 'company'
    )
);

```

4. PARCOURS UTILISATEURS

4.1 Programmeur - Réservation Sans Compte

Point d'entrée : Lien direct vers un spectacle

```
`/shows/le-malade-imaginaire`
```

Étapes :

1. **Arrivée sur la page spectacle** - Voir image, description, durée, compagnie, lieu - Voir le calendrier des représentations disponibles - Voir la disponibilité en temps réel (places restantes)
2. **Sélection d'un créneau** - Clic sur une date/heure dans le calendrier - Affichage du récapitulatif : date, heure, lieu, places disponibles - Bouton "Réserver cette représentation"

3. **Remplissage du formulaire** - Informations personnelles (prénom, nom, email, téléphone) - Informations professionnelles (fonction, structure, numéro AFC) - Nombre de places (dans la limite définie par le spectacle) - Commentaire optionnel - Acceptation des conditions
4. **Validation** - Vérification de la disponibilité - Création de la réservation - Affichage immédiat de la confirmation
5. **Confirmation** - Message "Réservation confirmée" - Code de réservation - Email envoyé avec détails - Événement Google Calendar ajouté à son agenda - Suggestions de spectacles similaires - Lien vers le catalogue complet

Durée estimée : 2-3 minutes

4.2 Programmeur - Avec Compte

Avantages :

- Profil pré-rempli
- Historique des réservations
- Gestion autonome (modification, annulation)

Parcours de création de compte :

1. **Inscription** - Email + mot de passe - OU Magic Link (lien OTP par email)
2. **Complétion du profil** - Toutes les infos professionnelles (une seule fois) - Préférences de notification
3. **Dashboard personnel** - Réservations à venir - Historique - Accès rapide au catalogue
4. **Réservation en 1 clic** - Sélection créneau - Nombre de places - Validation → Confirmé
5. **Gestion de ses réservations** - Modifier : changer créneau, nombre de places, ou infos - Annuler : avec motif optionnel - Toute modification met à jour Google Calendar automatiquement
6. **Suppression de compte** - Warnings multiples - Annulation automatique de toutes les réservations futures - Anonymisation des données

4.3 Admin Derviche - Gestion Quotidienne

1. Création d'un spectacle

- a. - Informations générales (titre, compagnie, description)
- b. - Catégories/genres (multi-select)
- c. - Image principale
- d. - Tarification (gratuit ou payant sur place)
- e. - Nombre max d'invitations par réservation - Informations pratiques

2. Programmation des représentations

- a. - Ajout de créneaux un par un
- b. - OU duplication en masse (ex: tous les jours à 18h du 10 au 20 juillet)
- c. - Pour chaque créneau :
- d. - Date, heure
- e. - Salle (peut varier)
- f. - Capacité
- g. - Accueil assuré par (Derviche ou Compagnie)

3. Publication

- a. - Génération automatique du slug et URL publique
- b. - Génération du QR Code (pour communication papier)
- c. - Publication → visible dans le catalogue

4. Gestion des réservations

- a. - Vue globale avec filtres (date, spectacle, compagnie, statut)
- b. - Recherche full-text
- c. - Création manuelle (réservation téléphonique)
- d. - Modification avec option de notification
- e. - Annulation avec motif
- f. - Export CSV

5. Accueil sur place (Check-in)

- a. - Interface mobile

- b. - Recherche rapide du programmeur
- c. - Sélection du statut :
 - i. - ❤️ Présent / A aimé
 - ii. - 📰 Présent / Presse
 - iii. - 😐 Présent / Neutre
 - iv. - ❌ Absent
- d. - Ajout de commentaires (accueil, salle)
- e. - Modification du profil si besoin - Notes internes

6. Statistiques

- a. - Filtres temporels (7j, 30j, période perso)
- b. - Nombre de réservations, taux de présence
- c. - Top spectacles
- d. - Export des données

4.4 Compagnie - Suivi de ses Spectacles

1. Dashboard

- a. - Vue d'ensemble de ses spectacles
- b. - Nombre de réservations par spectacle
- c. - Taux de remplissage
- d. - Taux de présence

2. Calendrier de ses représentations

- a. - Vue chronologique
- b. - Identification des créneaux où la compagnie accueille

3. Liste des réservations

- a. - Par spectacle et par créneau
- b. - Export CSV
- c. - Impression de liste d'émargement

4. Check-in (si accueil par la compagnie)

- a.
 - Mode simplifié
- b.
 - Tous les créneaux de leurs spectacles
- c.
 - Statut de présence (4 choix)
- d.
 - Commentaires d'accueil et infos salle
- e.
 - Modification du profil programmeur autorisée

5. FONCTIONNALITÉS PAR RÔLE

5.1 Authentification (Tous)

ID	Fonctionnalité	Description
F-AUTH-01	Connexion email/password	Connexion classique avec validation
F-AUTH-02	Magic Link	Connexion sans mot de passe (professionnels uniquement)
F-AUTH-03	Inscription	Création de compte programmeur
F-AUTH-04	Gestion de session	Maintien automatique, refresh token
F-AUTH-05	Déconnexion	Déconnexion globale, nettoyage local
F-AUTH-06	Récupération mot de passe	Email avec lien de réinitialisation

5.2 Derviche Diffusion (Admin)

5.2.1 Dashboard

ID	Fonctionnalité	Description
F-ADMIN-01	Dashboard global	Statistiques clés avec filtres temporels
F-ADMIN-02	Filtres période	7j, 30j, mois, année, période perso

ID	Fonctionnalité	Description
F-ADMIN-03	Métriques	Réservations, présence, spectacles, programmeurs
F-ADMIN-04	Graphiques	Évolution dans le temps
F-ADMIN-05	Top spectacles	Classement par réservations/présence

5.2.2 Gestion des Spectacles

ID	Fonctionnalité	Description
F-ADMIN-06	Créer spectacle	Formulaire complet (titre, desc, durée, compagnie, etc.)
F-ADMIN-07	Associer catégories	Multi-select des genres
F-ADMIN-08	Upload image	Image principale + galerie (Supabase Storage)
F-ADMIN-09	Générer slug	Auto depuis le titre (unique)
F-ADMIN-10	URL publique	<code>/shows/[slug]</code> générée automatiquement
F-ADMIN-11	Tarification	Gratuit ou payant sur place (montant)
F-ADMIN-12	Max invitations	Nombre max de places par réservation
F-ADMIN-13	Statut	Draft / Published / Archived
F-ADMIN-14	Modifier spectacle	Édition complète
F-ADMIN-15	Supprimer spectacle	Avec confirmation (cascade sur créneaux/résa)
F-ADMIN-16	Liste spectacles	Grille de cartes avec filtres
F-ADMIN-17	Recherche	Par titre, compagnie, genre

5.2.3 Gestion des Créneaux

ID	Fonctionnalité	Description
F-ADMIN-18	Créer créneau	Date, heure, capacité, salle, hosted_by
F-ADMIN-19	Duplication masse	Créer plusieurs créneaux d'un coup (ex: tous les jours à 18h)
F-ADMIN-20	Multi-salles	Un spectacle peut jouer dans différentes salles
F-ADMIN-21	Définir accueil	Derviche ou Compagnie par créneau
F-ADMIN-22	Modifier créneau	Édition (date, heure, capacité, salle)

ID	Fonctionnalité	Description
F-ADMIN-23	Supprimer créneau	Avec vérification des réservations existantes
F-ADMIN-24	Calcul auto capacité	remaining_capacity mis à jour automatiquement

5.2.4 Gestion des Réservations

ID	Fonctionnalité	Description
F-ADMIN-25	Liste réservations	Tableau responsive avec toutes les infos
F-ADMIN-26	Filtres avancés	Date, spectacle, compagnie, salle, statut
F-ADMIN-27	Filtres temporels	Période personnalisée
F-ADMIN-28	Recherche full-text	Nom, email, AFC, structure, téléphone
F-ADMIN-29	Tri colonnes	Tri ascendant/descendant
F-ADMIN-30	Créer réservation	Manuelle (ex: téléphone)
F-ADMIN-31	Modifier réservation	Créneau, places, infos programmeur
F-ADMIN-32	Option notification	Checkbox "Envoyer email au programmeur"
F-ADMIN-33	Annuler réservation	Avec motif + libération capacité
F-ADMIN-34	Export CSV	Avec filtres appliqués
F-ADMIN-35	Vue détaillée	Popup avec toutes les infos + historique
F-ADMIN-36	Date de réservation	Affichée dans la liste (created_at)

5.2.5 Check-in sur Place

ID	Fonctionnalité	Description
F-ADMIN-37	Mode check-in	Interface mobile-first
F-ADMIN-38	Filtres créneaux	Par spectacle, par date
F-ADMIN-39	Recherche rapide	Par nom, email, structure, AFC
F-ADMIN-40	4 statuts présence	Aimé / Presse / Neutre / Absent
F-ADMIN-41	Commentaire accueil	Notes prises pendant l'accueil

ID	Fonctionnalité	Description
F-ADMIN-42	Infos salle	Remarques sur les conditions du spectacle
F-ADMIN-43	Modification profil	Tous les champs éditables (mise à jour directe)
F-ADMIN-44	Notes internes	Visibles uniquement par Derviche
F-ADMIN-45	Stats temps réel	Nombre de présents/absents

5.2.6 Gestion des Entités

ID	Fonctionnalité	Description
F-ADMIN-46	CRUD Compagnies	Create, Read, Update, Delete
F-ADMIN-47	CRUD Salles	Create, Read, Update, Delete + GPS
F-ADMIN-48	CRUD Catégories	Create, Read, Update, Delete + ordre d'affichage
F-ADMIN-49	Réorganiser catégories	Drag & drop ou flèches haut/bas
F-ADMIN-50	Upload logos	Pour compagnies
F-ADMIN-51	Upload photos	Pour salles

5.2.7 Gestion des Utilisateurs (Super-Admin uniquement)

ID	Fonctionnalité	Description
F-ADMIN-52	Liste utilisateurs	Tous les rôles (admin voit tous sauf super-admin)
F-ADMIN-53	Filtrer par rôle	Super-Admin / Admin / Externe-DD / Professional / Company
F-ADMIN-54	Inviter membre	Email d'invitation (professional uniquement)
F-ADMIN-55	Créer compte compagnie	Associé à une compagnie existante (admin et super-admin)
F-ADMIN-55a	Créer compte admin	Super-admin uniquement
F-ADMIN-55b	Créer compte externe-DD	Avec mot de passe temporaire (super-admin uniquement)
F-ADMIN-55c	Assigner spectacles	Multi-select des spectacles pour externe-DD (super-admin uniquement)
F-ADMIN-55d	Modifier assignations	Ajouter/retirer des spectacles aux externe-DD (super-admin uniquement)
F-ADMIN-56	Modifier utilisateur	Changer rôle (selon permissions), infos
F-ADMIN-57	Désactiver compte	Soft delete

ID	Fonctionnalité	Description
F-ADMIN-58	Supprimer compte	Hard delete avec anonymisation

5.2.8 Paramètres & Configuration

ID	Fonctionnalité	Description
F-ADMIN-59	Nom organisation	Nom de l'organisation affiché sur la plateforme (par défaut "Derviche Pro", modifiable via app_settings)
F-ADMIN-60	Logo organisation	Upload + aperçu
F-ADMIN-61	Coordonnées	Email, téléphone, site web
F-ADMIN-62	Templates emails	Personnalisables
F-ADMIN-63	Config rappels	J-7, J-2, H-12 (activables)
F-ADMIN-64	Horaire envoi	Heure d'envoi des rappels
F-ADMIN-65	Google Calendar	Configuration API
F-ADMIN-66	Limites de rate limiting	Configuration des limites par endpoint (super-admin uniquement)

5.3 Programmeurs (Professionnels)

5.3.1 Découverte & Réservation

ID	Fonctionnalité	Description
F-PRO-01	Catalogue public	Page <code>/shows</code> avec tous les spectacles publiés
F-PRO-02	Filtres catalogue	Genre, date, lieu, horaire, disponibilité
F-PRO-03	Recherche front-end	Par titre, compagnie, description
F-PRO-04	Page spectacle	<code>/shows/[slug]</code> avec infos complètes
F-PRO-05	Calendrier réservation	Type Calendly, dispo temps réel
F-PRO-06	Sélection créneau	Clic sur date/heure disponible
F-PRO-07	Formulaire réservation	Infos perso + pro + commentaire
F-PRO-08	Choix nombre places	Dans la limite définie par spectacle

ID	Fonctionnalité	Description
F-PRO-09	Validation instantanée	Réservation confirmée immédiatement
F-PRO-10	Confirmation page	Récap + suggestions + lien catalogue
F-PRO-11	Email confirmation	Avec détails complets
F-PRO-12	Google Calendar	Événement ajouté automatiquement

5.3.2 Compte & Dashboard

ID	Fonctionnalité	Description
F-PRO-13	Création compte	Email/pwd ou Magic Link
F-PRO-14	Profil complet	Infos perso + pro (une fois)
F-PRO-15	Dashboard perso	Réservations à venir
F-PRO-16	Historique	Réservations passées
F-PRO-17	Réservation rapide	Infos pré-remplies, validation 1 clic
F-PRO-18	Modifier réservation	Créneau, places, ou infos
F-PRO-19	Annuler réservation	Avec motif optionnel
F-PRO-20	Suppression compte	Multi-warnings + anonymisation
F-PRO-21	Maj auto Calendar	Toute modif/annulation met à jour Google Calendar

5.4 Compagnies Artistiques

5.4.1 Consultation

ID	Fonctionnalité	Description
F-COMP-01	Dashboard compagnie	Stats de leurs spectacles uniquement
F-COMP-02	Vue calendrier	Toutes les représentations de leurs spectacles

ID	Fonctionnalité	Description
F-COMP-03	Niveau remplissage	Par créneau
F-COMP-04	Identification accueil	Voir qui accueille (Derviche ou eux)

5.4.2 Réservations

ID	Fonctionnalité	Description
F-COMP-05	Liste réservations	Par spectacle et par créneau
F-COMP-06	Détails réservations	Nom, structure, contacts
F-COMP-07	Export CSV	Listes d'émargement
F-COMP-08	Impression	Liste papier pour accueil

5.4.3 Check-in (si accueil par compagnie)

ID	Fonctionnalité	Description
F-COMP-09	Mode check-in simplifié	Par spectacle (tous créneaux)
F-COMP-10	Statut présence	4 choix (aimé/presse/neutre/absent)
F-COMP-11	Commentaires	Accueil + salle
F-COMP-12	Modification profil	Tous les champs éditables (mise à jour directe)

5.4.4 Statistiques

ID	Fonctionnalité	Description
F-COMP-13	Taux remplissage	Par spectacle
F-COMP-14	Taux présence	Évolution
F-COMP-15	Profil programmeurs	Fonction, région

5.5 Externes-DD (Externe Derviche Diffusion)

5.5.1 Dashboard

ID	Fonctionnalité	Description
F-EXT-01	Dashboard personnalisé	Stats uniquement des spectacles assignés
F-EXT-02	Filtres période	7j, 30j, période personnalisée

ID	Fonctionnalité	Description
F-EXT-03	Métriques limitées	Réservations, présence de leurs spectacles uniquement
F-EXT-04	Liste spectacles assignés	Vue des spectacles dont ils ont la charge

5.5.2 Réservations

ID	Fonctionnalité	Description
F-EXT-05	Liste réservations filtrée	Uniquement spectacles assignés
F-EXT-06	Filtres avancés	Date, spectacle (assignés), salle, statut
F-EXT-07	Recherche full-text	Nom, email, AFC, structure, téléphone
F-EXT-08	Créer réservation	Manuelle (spectacles assignés uniquement)
F-EXT-09	Modifier réservation	Créneau, places, infos (spectacles assignés)
F-EXT-10	Option notification	Checkbox "Envoyer email au programmeur"
F-EXT-11	Annuler réservation	Avec motif (spectacles assignés)
F-EXT-12	Export CSV	Avec filtres appliqués (spectacles assignés)
F-EXT-13	Vue détaillée	Popup avec toutes les infos + historique
F-EXT-14	Accès notes internes	Lecture, création, modification

5.5.3 Check-in

ID	Fonctionnalité	Description
F-EXT-15	Mode check-in mobile	Interface mobile-first
F-EXT-16	Filtres créneaux	Spectacles assignés uniquement, par date
F-EXT-17	Recherche rapide	Par nom, email, structure, AFC
F-EXT-18	4 statuts présence	Aimé / Presse / Neutre / Absent
F-EXT-19	Commentaire accueil	Notes prises pendant l'accueil

ID	Fonctionnalité	Description
F-EXT-20	Infos salle	Remarques sur les conditions du spectacle
F-EXT-21	Modification profil	Tous les champs éditables (mise à jour directe)
F-EXT-22	Notes internes	Création et modification autorisées
F-EXT-23	Stats temps réel	Nombre de présents/absents (leurs spectacles)

5.5.4 Consultation

ID	Fonctionnalité	Description
F-EXT-24	Vue compagnies	Lecture seule
F-EXT-25	Vue salles	Lecture seule
F-EXT-26	Vue catégories	Lecture seule
F-EXT-27	Calendrier créneaux	Tous les créneaux de leurs spectacles assignés

6. RÈGLES MÉTIER

6.1 Spectacles

Règle	Description
R-SHOW-01	Un spectacle doit avoir au moins une catégorie
R-SHOW-02	Le slug est généré automatiquement à partir du titre et doit être unique
R-SHOW-03	Un spectacle ne peut être supprimé s'il a des créneaux avec des réservations confirmées
R-SHOW-04	Un spectacle peut être dans plusieurs salles (via les créneaux)
R-SHOW-05	max_reservations_per_booking doit être entre 1 et 10
R-SHOW-06	Un spectacle "draft" n'est pas visible publiquement
R-SHOW-07	Un spectacle "archived" reste visible dans l'admin mais pas dans le catalogue

6.2 Créneaux

Règle	Description
R-SLOT-01	remaining_capacity ne peut jamais être négatif

Règle	Description
R-SLOT-02	remaining_capacity ne peut jamais dépasser capacity
R-SLOT-03	Un créneau ne peut être supprimé s'il a des réservations confirmées
R-SLOT-04	La date d'un créneau doit être dans le futur (lors de la création)
R-SLOT-05	hosted_by détermine qui peut faire le check-in

6.3 Réservations

Règle	Description
R-RESA-01	Une réservation ne peut être créée que si remaining_capacity >= num_places
R-RESA-02	num_places doit respecter max_reservations_per_booking du spectacle
R-RESA-03	Un email peut avoir plusieurs réservations (créneaux différents)
R-RESA-04	Un email ne peut avoir qu'une seule réservation par créneau
R-RESA-05	La modification d'une réservation met à jour Google Calendar
R-RESA-06	L'annulation d'une réservation supprime l'événement Google Calendar
R-RESA-07	Une réservation annulée libère la capacité du créneau
R-RESA-08	Le check-in ne peut se faire que le jour J ou après

6.4 Google Calendar

Règle	Description
R-CAL-01	Un événement est créé en arrière-plan (queue)
R-CAL-02	L'événement est ajouté au calendrier Derviche Diffusion
R-CAL-03	Les invités sont : programmateur, compagnie (si hosted_by=company), Derviche
R-CAL-04	Les rappels sont configurables (J-7, J-2, H-12)
R-CAL-05	Chaque réservation = 1 événement séparé
R-CAL-06	L'ID de l'événement est stocké dans google_calendar_event_id
R-CAL-07	En cas d'échec de création, retry automatique (3 fois)

6.5 Emails

Règle	Description
R-EMAIL-01	Email de confirmation envoyé immédiatement après réservation
R-EMAIL-02	Email de modification envoyé si demandé par admin
R-EMAIL-03	Email d'annulation toujours envoyé
R-EMAIL-04	Rappels automatiques configurables
R-EMAIL-05	Tous les emails sont loggés dans sent_notifications

6.6 Check-in

Règle	Description
R-CHECKIN-01	4 statuts possibles : present_loved, present_press, present_neutral, absent
R-CHECKIN-02	Le check-in modifie directement le profil du programmeur
R-CHECKIN-03	Les compagnies ne peuvent faire le check-in que si hosted_by=company
R-CHECKIN-04	Les compagnies peuvent modifier le profil des programmeurs pendant le check-in
R-CHECKIN-05	Les notes internes sont visibles uniquement par Derviche (super-admin, admin, externe-dd)
R-CHECKIN-06	Les externes-DD peuvent faire le check-in sur les créneaux de leurs spectacles assignés

6.7 Catégories

Règle	Description
R-CAT-01	Une catégorie ne peut être supprimée si des spectacles y sont associés
R-CAT-02	Le slug de catégorie doit être unique
R-CAT-03	L'ordre d'affichage détermine l'ordre dans les filtres

6.8 Utilisateurs

Règle	Description
R-USER-01	Un utilisateur ne peut avoir qu'un seul rôle
R-USER-02	Seuls les super-admin peuvent créer des comptes admin, externe-dd ou company. Les admins peuvent créer des comptes company uniquement.

Règle	Description
R-USER-03	Les programmeurs peuvent s'inscrire eux-mêmes
R-USER-04	La suppression d'un compte annule toutes ses réservations futures
R-USER-05	La suppression d'un compte anonymise les données (RGPD)

6.9 Rôles et Permissions

Règle	Description
R-ROLE-01	Il existe 5 rôles : super-admin, admin, professional, company, externe-dd
R-ROLE-02	Seuls les super-admin peuvent créer/modifier/supprimer des comptes admin et externe-dd
R-ROLE-03	Les admins ne peuvent pas créer d'autres admins ni des externes-dd
R-ROLE-04	Les super-admin peuvent gérer les assignations spectacles des externes-dd
R-ROLE-05	Un externe-dd doit avoir au moins 1 spectacle assigné pour être actif

6.10 Externes-DD

Règle	Description
R-EXT-01	Un externe-dd ne voit que les spectacles qui lui sont assignés
R-EXT-02	Un externe-dd peut faire le check-in sur tous les créneaux de ses spectacles assignés où hosted_by='derviche'
R-EXT-03	Un externe-dd peut modifier le profil des programmeurs pendant le check-in
R-EXT-04	Un externe-dd peut créer/modifier/annuler des réservations uniquement pour ses spectacles assignés
R-EXT-05	Un externe-dd peut voir, créer et modifier les notes internes
R-EXT-06	Un externe-dd a accès aux infos de contact complètes des programmeurs
R-EXT-07	Un externe-dd peut exporter en CSV les réservations de ses spectacles assignés
R-EXT-08	Un externe-dd ne peut pas créer/modifier des spectacles, créneaux, compagnies ou salles

7. INTERFACES & WIREFRAMES

7.1 Page Catalogue Public `/shows`

HEADER [Logo] [Connexion]			
PROGRAMMATION 2025 42 Spectacles à découvrir			
FILTRES			
Genre: [Tous ▼] Date: [Juillet ▼]			
Lieu: [Tous ▼] Horaire: [Tous ▼]			
[✓] Seulement disponibles			
 [Rechercher...]			
[IMG]	[IMG]	[IMG]	[IMG]
Titre	Titre	Titre	Titre
Cie	Cie	Cie	Cie
★★★★★	★★★★★	★★★★★	★★★★★
12 pl	5 pl	Complet	8 pl
[Voir]	[Voir]	[Voir]	[Voir]
[Pagination ou Infinite Scroll]			

7.2 Page Spectacle `/shows/[slug]`

--

[\[← Retour au catalogue\]](#)

IMAGE HERO DU SPECTACLE

LE MALADE IMAGINAIRE

Par Compagnie du Rire

Théâtre des Carmes

 Théâtre classique  1h30  Gratuit

DESCRIPTION

[Texte complet de la description...]

 17 RÉSERVER UNE REPRÉSENTATION

Juillet 2025

Lu Ma Me Je Ve Sa Di

1 2 3 4 5 6 7

8 9 [10][11][12] 13 14

[10] = 14h00  12 places

[11] = 18h00  3 places

[12] = 14h00  Complet

[Créneaux suivants si défilement...]

[Si créneau sélectionné → Formulaire de réservation]

7.3 Dashboard Admin `/admin`

DERVICHE DIFFUSION |

[Dashboard] [Spectacles] [Résa] [Paramètres] |

TABLEAU DE BORD |

Période : [7 derniers jours ▼] [01/07 - 31/07] |



1,247

Résa



78%

Présent



42

Spect.



18

Salles

ÉVOLUTION DES RÉSERVATIONS |

[Graphique linéaire] |

TOP 5 SPECTACLES |

1. Le Malade Imaginaire - 124 réservations |

2. Hamlet - 98 réservations |

3. Tartuffe - 87 réservations |

... |

ALERTES |

⚠ 3 spectacles proches de la complétude |

⚠ 5 réservations modifiées aujourd'hui |

7.4 Liste Réservations Admin `/admin/reservations`

GESTION DES RÉSERVATIONS

Filtres :

July 17

[01/07 - 31/07]

👤

[Tous ▼]

🏛️

[Tous ▼]

📊

[Tous statuts ▼]

🔍

[Rechercher...]

📄

Export CSV

+

Nouvelle réservation

1,247 résultats

Spectacle	Programmateur	Date	Statut
Le Malade...	Jean Dupont	10/07	✅ Confirmé
2 places	Théâtre X	14h00	
👁️🖋️❌✉️			
Hamlet	Marie Martin	15/07	⌚ Attente
1 place	Salle Y	18h00	
👁️🖋️❌✉️			
...	...	...	...

7.5 Mode Check-in Mobile `/admin/checkin`

 CHECK-IN
Le Malade Imaginaire
10/07/2025 - 14h00
Théâtre des Carmes
 12/15 confirmés
☒ 8 présents ☒ 0 absents
 [Recherche rapide...]
LISTE
☒ Jean Dupont (2 pl)
Théâtre X • AFC 12345
Arrivé à 13:47
[Voir détails]
☐ Marie Martin (1 pl)
Salle Y • AFC 67890
Statut : [Sélectionner ▼]
()  Présent / A aimé
()  Présent / Presse
()  Présent / Neutre
() ☒ Absent
[Ajouter notes]
[ Modifier profil]
...

7.6 Dashboard Programmeur `/professional`

Bonjour Jean Dupont 🖐️ |

MES RÉSERVATIONS À VENIR (3) |

 17 10 juillet 2025 - 14h00 | |
LE MALADE IMAGINAIRE | |
Théâtre des Carmes • 2 places | |
[Voir] [✎ Modifier] [✖ Annuler] | |

 17 15 juillet 2025 - 18h00 | |
HAMLET | |
Théâtre du Chêne Noir • 1 place | |
[Voir] [✎ Modifier] [✖ Annuler] | |

[🔍👉 Découvrir d'autres spectacles] |

HISTORIQUE (5) |
[Voir toutes mes réservations passées] |

MON COMPTE |
[⚙️ Modifier mes informations] |

[🗑️ Supprimer mon compte] |

7.7 Dashboard Externe-DD /externe-dd

DERVICHE DIFFUSION

[Dashboard] [Mes spectacles] [Réservations] [✓] |

TABLEAU DE BORD - Jean (Externe)

📋 Mes spectacles assignés : 5

Période : [7 derniers jours ▼] [01/07 - 31/07] |



127

Résa



82%

Présent



15

Créneaux



MES SPECTACLES

Le Malade Imaginaire - 42 réservations

[📊 Stats] [📋 Réservations] [✓ Check-in] |

Hamlet - 35 réservations

[📊 Stats] [📋 Réservations] [✓ Check-in] |



PROCHAINS CRÉNEAUX (Check-in à venir)

• 10/07 - 14h00 - Le Malade Imaginaire (12 résa) |

- | | |
|--------------------------------------|--|
| • 10/07 - 18h00 - Hamlet (8 résa) | |
| • 11/07 - 14h00 - Tartuffe (15 résa) | |
| | |
| [🎯] Accéder au mode Check-in | |

8. SÉCURITÉ & RGPD

8.1 Authentification & Sécurité

Authentification :

- Supabase Auth (JWT tokens)
- Sessions avec refresh automatique
- Magic Link avec expiration (1h)

Rate Limiting :

Limites généreuses pour le MVP, **configurables par le super-admin** via l'interface de paramètres :

🔑 Authentification (`/api/auth/*`)

- **Login** : 10 tentatives / 15 min par IP
- **Magic Link** : 6 demandes / heure par email
- **Récupération mot de passe** : 6 demandes / heure par email

📅 Réservations (`/api/reservations`)

- **POST (création)** : 20 réservations / 10 min par utilisateur
- **PUT (modification)** : 40 modifications / heure par utilisateur
- **DELETE (annulation)** : 40 annulations / heure par utilisateur

✉️ Emails (envois manuels admin)

- **Notifications manuelles** : 100 emails / heure par admin

🔍 Recherche/Catalogue publique

- **GET /api/shows** : 200 req / minute par IP

- **GET /shows/[slug]** : 400 req / minute par IP

Admin - Opérations en masse

- **Création/Suppression** : 100 opérations / minute
- **Export CSV** : 20 exports / heure par utilisateur

Implémentation technique :

```
// Avec Upstash Redis (déjà dans la stack)
import { Ratelimit } from '@upstash/ratelimit';
import { Redis } from '@upstash/redis';

// Exemple : Limite pour création de réservations
const ratelimit = new Ratelimit({
  redis: Redis.fromEnv(),
  limiter: Ratelimit.slidingWindow(20, '10 m'),
});
```

8.2 Autorisation

- Row Level Security (RLS) sur toutes les tables
- Vérification du rôle côté serveur pour toute action
- Fonctions `has_role()` et `is_admin()`
- Middleware Next.js pour protéger les routes

8.3 Données Personnelles (RGPD)

Données collectées

- **Obligatoires** : prénom, nom, email, téléphone, fonction, structure
- **Optionnelles** : email2, phone2, adresse, AFC, commentaires

Finalités

- Gestion des réservations
- Communication liée à la programmation de spectacles
- Statistiques anonymisées

Droits des utilisateurs

- **Accès** : Dashboard avec toutes les infos
- **Rectification** : Modification du profil
- **Suppression** : Bouton "Supprimer mon compte"
- **Portabilité** : Export CSV (admin)

Conservation

- Réservations actives : Durée de la programmation + 1 an
- Réservations annulées : Anonymisées après 3 mois
- Comptes supprimés : Anonymisation immédiate

Sécurité technique

- Chiffrement des données en transit (HTTPS)
- Chiffrement des données au repos (Supabase)
- Backups quotidiens
- Logs d'accès (audit_logs)

8.4 Conformité

- ☒ Mentions légales
 - ☒ Politique de confidentialité
 - ☒ CGU
 - ☒ Banner de consentement cookies (si analytics)
-

9. ROADMAP & PRIORISATION

9.1 Phase 1 : MVP (13 semaines)

Sprint 1-2 : Fondations (2 sem)

- ☐ Setup Next.js 14 + TypeScript + Tailwind
- ☐ Configuration Supabase (DB + Auth + Storage)
- ☐ Configuration Upstash Redis

- ☐ Setup BullMQ (workers)
- ☐ Google Calendar API (credentials + tests)
- ☐ Configuration Resend (emails)
- ☐ Auth email/password + magic link
- ☐ Système de rôles (RLS)
- ☐ Layouts par rôle
- ☐ Navigation & routing

Sprint 3 : Admin - Entités (1 sem)

- ☐ CRUD Compagnies
- ☐ CRUD Salles (+ GPS)
- ☐ CRUD Catégories
- ☐ Upload d'images (Storage)

Sprint 4 : Admin - Spectacles (1 sem)

- ☐ CRUD Spectacles (formulaire complet)
- ☐ Génération slug
- ☐ Association catégories (multi-select)
- ☐ Statut (draft/published/archived)
- ☐ URL publique générée

Sprint 5 : Admin - Créneaux (1 sem)

- ☐ Création manuelle de créneaux
- ☐ Duplication en masse
- ☐ Sélection venue par créneau
- ☐ Définir hosted_by

Sprint 6-7 : Réservation Publique (2 sem)

- ☐ Page catalogue `/shows`
- ☐ Filtres + recherche front-end

- ☐ Page spectacle `/shows/[slug]`
- ☐ Calendrier type Calendly
- ☐ Formulaire de réservation (+ commentaire)
- ☐ API POST `/api/reservations`
- ☐ Workers BullMQ (Calendar + Emails)
- ☐ Page de confirmation
- ☐ Suggestions de spectacles

Sprint 8 : Admin - Réservations (1 sem)

- ☐ Liste avec filtres
- ☐ Création manuelle
- ☐ Modification (+ option notification)
- ☐ Annulation
- ☐ Export CSV

Sprint 9-10 : Dashboard Pro (2 sem)

- ☐ Inscription/Connexion
- ☐ Profil complet
- ☐ Dashboard avec réservations
- ☐ Modification de réservation
- ☐ Annulation de réservation
- ☐ Suppression de compte

Sprint 11 : Check-in + Externes-DD (1,5 sem)

- ☐ Mode check-in admin (mobile-first)
- ☐ 4 statuts + commentaires
- ☐ Modification profil
- ☐ Notes internes (création/modification)
- ☐ Mode check-in compagnie (limité)
- ☐ **Rôle externe-DD**

- ☐ Table user_show_assignments
- ☐ Fonctions RLS externes-DD
- ☐ UI création compte externe-DD (super-admin)
- ☐ UI assignation spectacles
- ☐ Dashboard externe-DD
- ☐ Mode check-in externe-DD (avec modif profil)
- ☐ Filtrage automatique des données (spectacles assignés)

Sprint 12-13 : Finitions (2 sem)

- ☐ Dashboard admin avec filtres temporels
- ☐ Stats & graphiques
- ☐ Paramètres (nom orga, logo, etc.)
- ☐ Gestion utilisateurs
- ☐ Tests utilisateurs
- ☐ Corrections bugs
- ☐ Optimisations
- ☐ Documentation

LIVRAISON MVP : Semaine 13,5

9.2 Phase 2 : Post-MVP (optionnel)

9.2 Phase 2 : White-Label & Multi-Tenant (4 semaines)

 **Objectif** : Transformer la plateforme en solution SaaS vendable à d'autres organisations

Sprint 14 : Architecture Multi-Tenant (1 sem)

Base de données :

- ☐ Créer table `tenants` avec toutes les métadonnées
- ☐ Ajouter `tenant_id` à toutes les tables existantes
- ☐ Créer le tenant Derviche Diffusion

- ☐ Migrer toutes les données existantes vers ce tenant
- ☐ Ajouter les index de performance

Sécurité :

- ☐ Mettre à jour toutes les fonctions RLS
- ☐ Créer les politiques d'isolation par tenant
- ☐ Tests d'isolation rigoureux (2+ tenants fictifs)
- ☐ Fonction `get_current_tenant_id()`

Backend :

- ☐ Middleware Next.js pour détection tenant
- ☐ Support sous-domaines (tenant.plateforme.com)
- ☐ Context API tenant côté client
- ☐ Tests end-to-end multi-tenant

Sprint 15 : Personnalisation & Branding (1 sem)

Interface d'admin tenant :

- ☐ Page `/tenant-admin` (super-admin uniquement)
- ☐ Section Branding
 - ☐ Upload logo + preview
 - ☐ Color picker (primaire, secondaire, accent)
 - ☐ Upload favicon
- ☐ Section Configuration
 - ☐ Nom organisation (éditable)
 - ☐ Contacts (email, téléphone, site)
 - ☐ Fuseau horaire
 - ☐ Langue par défaut

Application des styles :

- ☐ Thème dynamique basé sur tenant
- ☐ CSS variables injectées

☐ Logo affiché dans header

☐ Favicon dynamique

Emails white-label :

☐ Templates avec logo tenant

☐ Couleurs personnalisables

☐ Nom organisation dans subject/body

Sprint 16 : Domaines Custom & Provisioning (1 sem)

Domaines personnalisés :

☐ Support CNAME (reservations.client.com)

☐ Configuration DNS automatique (si possible)

☐ Documentation pour les clients

☐ Certificats SSL automatiques (Let's Encrypt via Vercel)

Provisioning automatique :

☐ API POST /api/tenants (création)

☐ Workflow d'onboarding

☐ Création tenant

☐ Invitation super-admin

☐ Email de bienvenue

☐ Checklist d'onboarding

☐ Données de démo optionnelles

Gestion des limites :

☐ Vérification limites selon plan

☐ Max spectacles

☐ Max admins

☐ Max externes

☐ Soft limits avec alertes

☐ Hard limits avec blocage

Sprint 17 : Interface Commerciale & Facturation (1 sem)

Landing page de vente :

- ☐ Page d'accueil produit
- ☐ Page pricing avec 3 plans
- ☐ Page features détaillée
- ☐ Formulaire d'essai gratuit
- ☐ Témoignage Derviche Diffusion

Système d'inscription :

- ☐ Formulaire self-service
- ☐ Période d'essai 14 jours
- ☐ Email de bienvenue automatique
- ☐ Onboarding guidé

Facturation (Stripe) :

- ☐ Intégration Stripe Billing
- ☐ Gestion des plans (Starter, Pro, Enterprise)
- ☐ Factures automatiques
- ☐ Webhook gestion des paiements
- ☐ Interface de changement de plan
- ☐ Gestion des cartes bancaires

Dashboard de gestion tenant :

- ☐ Vue utilisation (spectacles, admins, etc.)
 - ☐ Indicateurs vs limites
 - ☐ Historique des factures
 - ☐ Prochaine facturation
-

9.3 Phase 3 : Post-MVP (optionnel)

Fonctionnalités Nice-to-Have

- ☐ Templates emails personnalisables (UI)

- ☐ Rappels automatiques configurables
 - ☐ Permissions granulaires (Super Admin, Modérateur, etc.)
 - ☐ Système de favoris programmeurs
 - ☐ Notifications push (PWA)
 - ☐ Mode offline pour check-in
 - ☐ Recherche PostgreSQL full-text (si > 100 spectacles)
 - ☐ Analytics avancées (Google Analytics, Mixpanel)
 - ☐ Export PDF des rapports
 - ☐ Multi-langues (EN/FR)
 - ☐ API publique pour partenaires
 - ☐ Intégration CRM
-

10. ANNEXES TECHNIQUES

10.1 Variables d'Environnement

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://xxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJxxx...
SUPABASE_SERVICE_ROLE_KEY=eyJxxx...

# Upstash Redis (BullMQ)
UPSTASH_REDIS_REST_URL=https://xxx.upstash.io
UPSTASH_REDIS_REST_TOKEN=xxx

# Google Calendar API
GOOGLE_CALENDAR_CLIENT_ID=xxx.apps.googleusercontent.com
GOOGLE_CALENDAR_CLIENT_SECRET=xxx
GOOGLE_CALENDAR_REDIRECT_URI=https://derviche-pro.vercel.app/api/auth/google/callback
GOOGLE_CALENDAR_REFRESH_TOKEN=xxx
GOOGLE_CALENDAR_ID=primary

# Resend (Emails)
```

```
RESEND_API_KEY=re_xxx
RESEND_FROM_EMAIL=noreply@derviche-pro.com

# Next.js
NEXT_PUBLIC_APP_URL=https://derviche-pro.vercel.app
NODE_ENV=production

# Optional: Analytics
NEXT_PUBLIC_GA_ID=G-XXXXXXXXXX
```

10.2 Commandes Utiles

```
# Développement local
npm run dev

# Build production
npm run build

# Démarrer production
npm start

# Linter
npm run lint

# Type checking
npm run type-check

# Migrations Supabase
npx supabase db reset
npx supabase db push
npx supabase db pull

# Tests (si configurés)
npm test
npm run test:e2e
```

```
# Générer les types Supabase
npx supabase gen types typescript --local > types/database.types.ts
```

10.3 Configuration BullMQ

```
// lib/queue.ts
import { Queue, Worker } from 'bullmq';
import Redis from 'ioredis';

const connection = new Redis(process.env.UPSTASH_REDIS_REST_URL!, {
  maxRetriesPerRequest: null,
});

// Queue pour les tâches liées aux réservations
export const reservationQueue = new Queue('reservations', { connection
});

// Worker pour traiter les jobs
export const reservationWorker = new Worker(
  'reservations',
  async (job) => {
    const { type, data } = job.data;

    switch (type) {
      case 'createGoogleCalendarEvent':
        await createCalendarEvent(data);
        break;
      case 'updateGoogleCalendarEvent':
        await updateCalendarEvent(data);
        break;
      case 'deleteGoogleCalendarEvent':
        await deleteCalendarEvent(data);
        break;
      case 'sendConfirmationEmail':
        await sendEmail('confirmation', data);
        break;
      case 'sendModificationEmail':
        await sendEmail('modification', data);
    }
  }
);
```

```

        break;
    case 'sendCancellationEmail':
        await sendEmail('cancellation', data);
        break;
    case 'sendReminderEmail':
        await sendEmail('reminder', data);
        break;
    default:
        throw new Error(`Unknown job type: ${type}`);
    }
},
{
    connection,
    concurrency: 5,
    limiter: {
        max: 10,
        duration: 1000,
    },
}
);

// Gestion des erreurs
reservationWorker.on('failed', (job, err) => {
    console.error(`Job ${job?.id} failed:`, err);
});

```

10.4 Types TypeScript Principaux

```

// types/index.ts

export type UserRole = 'super-admin' | 'admin' | 'professional' | 'company'
| 'externe-dd';

export type ShowStatus = 'draft' | 'published' | 'archived';

export type ReservationStatus = 'confirmed' | 'cancelled' | 'no_show';

export type CheckinStatus = 'present_loved' | 'present_press' | 'present_ne

```

```

utral' | 'absent';

export type HostedBy = 'derviche' | 'company';

export interface Profile {
  id: string;
  email: string;
  first_name: string | null;
  last_name: string | null;
  phone: string | null;
  email2: string | null;
  phone2: string | null;
  function: string | null;
  structure: string | null;
  afc_number: string | null;
  address: string | null;
  comments: string | null;
  company_id: string | null;
  created_at: string;
  updated_at: string;
}

export interface Show {
  id: string;
  slug: string;
  title: string;
  company_id: string;
  short_description: string | null;
  long_description: string | null;
  duration_minutes: number | null;
  image_url: string | null;
  gallery_urls: string[] | null;
  status: ShowStatus;
  price_type: 'free' | 'paid_on_site';
  price_amount: number | null;
  max_reservations_per_booking: number;
  practical_info: string | null;
  created_at: string;

```

```

    updated_at: string;
}

export interface Slot {
    id: string;
    show_id: string;
    venue_id: string;
    date: string;
    time: string;
    capacity: number;
    remaining_capacity: number;
    hosted_by: HostedBy;
    created_at: string;
    updated_at: string;
}

export interface Reservation {
    id: string;
    slot_id: string;
    user_id: string | null;
    guest_first_name: string | null;
    guest_last_name: string | null;
    guest_email: string | null;
    guest_phone: string | null;
    guest_function: string | null;
    guest_structure: string | null;
    guest_afc_number: string | null;
    num_places: number;
    status: ReservationStatus;
    special_requests: string | null;
    checkin_status: CheckinStatus | null;
    checkin_comment: string | null;
    checkin_venue_notes: string | null;
    checkin_internal_notes: string | null;
    checkin_at: string | null;
    checkin_by: string | null;
    google_calendar_event_id: string | null;
    created_at: string;
}

```

```
updated_at: string;
cancelled_at: string | null;
cancellation_reason: string | null;
}
```

10.5 Exemple de Worker Email

```
// workers/emailWorker.ts
import { Resend } from 'resend';

const resend = new Resend(process.env.RESEND_API_KEY);

interface EmailData {
  reservationId: string;
  recipientEmail: string;
  recipientName: string;
  showTitle: string;
  showDate: string;
  showTime: string;
  venueName: string;
  numPlaces: number;
}

export async function sendEmail(type: string, data: EmailData) {
  const { recipientEmail, recipientName, showTitle, showDate, showTime, venueName, numPlaces } = data;

  let subject: string;
  let html: string;

  switch (type) {
    case 'confirmation':
      subject = `Confirmation de réservation - ${showTitle}`;
      html = `
        <h1>Réservation confirmée</h1>
        <p>Bonjour ${recipientName},</p>
        <p>Votre réservation pour <strong>${showTitle}</strong> est confirmée.</p>
      `;

```



```

<h2>Détails :</h2>
<ul>
  <li><strong>Date :</strong> ${showDate}</li>
  <li><strong>Heure :</strong> ${showTime}</li>
  <li><strong>Lieu :</strong> ${venueName}</li>
  <li><strong>Nombre de places :</strong> ${numPlaces}</li>
</ul>
<p>À très bientôt !</p>
`;
break;

case 'modification':
  subject = `Modification de réservation - ${showTitle}`;
  html = `
    <h1>Réservation modifiée</h1>
    <p>Bonjour ${recipientName},</p>
    <p>Votre réservation pour <strong>${showTitle}</strong> a été modif
    iée.</p>
    <h2>Nouveaux détails :</h2>
    <ul>
      <li><strong>Date :</strong> ${showDate}</li>
      <li><strong>Heure :</strong> ${showTime}</li>
      <li><strong>Lieu :</strong> ${venueName}</li>
      <li><strong>Nombre de places :</strong> ${numPlaces}</li>
    </ul>
    `;
  break;

case 'cancellation':
  subject = `Annulation de réservation - ${showTitle}`;
  html = `
    <h1>Réservation annulée</h1>
    <p>Bonjour ${recipientName},</p>
    <p>Votre réservation pour <strong>${showTitle}</strong> a été annul
    ée.</p>
    <p>Si vous avez des questions, n'hésitez pas à nous contacter.</p>
    `;
  break;

```

```

    default:
      throw new Error(`Unknown email type: ${type}`);
    }

    const result = await resend.emails.send({
      from: process.env.RESEND_FROM_EMAIL!,
      to: recipientEmail,
      subject,
      html,
    });

    // Logger l'envoi dans sent_notifications
    await logNotification(data.reservationId, type, recipientEmail, result.id);

    return result;
  }

  async function logNotification(
    reservationId: string,
    type: string,
    recipientEmail: string,
    emailProviderId: string | null
  ) {
    // Implémenter l'insertion dans la table sent_notifications
  }

```

RÉSUMÉ EXÉCUTIF

Objectif

Plateforme de réservation professionnelle permettant aux programmeurs de théâtre de réserver gratuitement des places pour découvrir des spectacles tout au long de l'année.

Utilisateurs

- **2-3** super-administrateurs Derviche Diffusion

- **3-7** administrateurs Derviche Diffusion
- **10-20** externes Derviche Diffusion
- **500-1000** programmeurs professionnels
- **15-20** compagnies artistiques

Fonctionnalités Clés

- ✓ Catalogue public de spectacles avec réservation en ligne
- ✓ Gestion complète admin (spectacles, créneaux, réservations, check-in)
- ✓ Dashboard programmeur avec gestion autonome
- ✓ Intégration Google Calendar automatique
- ✓ Check-in mobile avec qualification (4 statuts)
- ✓ Statistiques avec filtres temporels
- ✓ White-label (nom organisation personnalisable)

Stack Technique

Next.js 14 • TypeScript • Supabase • BullMQ • Google Calendar API • Resend • Vercel

Durée MVP

13,5 semaines (~3 mois et demi)

Budget Estimé

- Supabase : ~\$25/mois (Pro plan)
 - Upstash Redis : ~\$10/mois (Pay as you go)
 - Resend : ~\$20/mois (jusqu'à 10k emails)
 - Vercel : Gratuit (hobby) ou \$20/mois (Pro)
 - **Total** : ~\$55-75/mois
-

  Stratégie White-Label & Multi-Tenant

Scripts SQL - Migration Multi-Tenant